Phishing Website Detection System with Multi-Layer AI and Blockchain-Based Threat Logging

A Project Report

Submitted by:

Ayush Sa, 2241019002,
Dheeraj Kumar, 2241011127,
Sreyanshu Epari, 2241005001,
Chirag Agrawal, 2241001038,
Pranit Kunar Mitra, 2241013199

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(Specialization in Cyber Security)

CENTRE FOR CYBER SECURITY
Department of Computer Science and Engineering

Faculty of Engineering & Technology
Institute of Technical Education and Research

Siksha 'O' Anusandhan (Deemed to be University)
Bhubaneswar, Odisha, India
(June 2026)

CERTIFICATE

This is to certify that the project report titled "**Phishing Website Detection System with Multi-Layer AI and Blockchain-Based Threat Logging**" being submitted by **Ayush Sa, Pranit Kumar Mitra, Sreyanshu Epari, Chirag Agrawal, and Dheeraj Kumar** of **Section – 2241024** to the **Institute of Technical Education and Research, Siksha 'O' Anusandhan (Deemed to be University)**, Bhubaneswar for the partial fulfillment for the degree of **Bachelor of Technology in Computer Science and Engineering (Specialization in Cyber Security)** is a record of original confide work carried out by them under our supervision and guidance. The project work, in my opinion, has reached the requisite standard fulfilling the requirements for the Degree of **Bachelor of Technology**.

The results contained in this project work have not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Dr. Pubali Chatterjee
Assistant Professor
Centre for Cyber Security
Department of Computer Science & Engineering
Faculty of Engineering & Technology, Institute of Technical Education and Research
Siksha 'O' Anusandhan (Deemed to be University), Bhubaneswar

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and deep sense of respect to our project supervisor, **Dr. Pubali Chatterjee** , Associate Professor, Centre for Cyber Security, Department of Computer Science and Engineering, Faculty of Engineering (ITER), Bhubaneswar, for his constant guidance, valuable suggestions, encouragement, and continuous support throughout the course of this project. His expertise in the fields of cyber security and machine learning has been an immense source of inspiration and learning for us. His scholarly supervision, technical insights, and constructive feedback greatly contributed to the successful completion of this research work.

I am deeply thankful to **Dr. Suneeta Satapathy**, Project Coordinator, for her constant motivation, valuable advice, and support throughout the project. I would also like to extend my heartfelt gratitude to **Dr. Bharat Jyoti Ranjan Sahu**, Chief Coordinator, Centre for Cybersecurity, for providing the necessary resources, academic environment, and encouragement required for carrying out this project successfully. Their guidance and encouragement have been instrumental in helping me achieve the objectives of this work.

I am grateful to all the faculty members and staff of the Centre for Cybersecurity and Department of Computer Science and Engineering for their cooperation and assistance whenever required. Their knowledge, support, and encouragement have significantly contributed to my learning experience.

I would also like to thank my classmates, friends, and family members for their unwavering support, motivation, and understanding throughout the project period.

Finally, I express my sincere appreciation to everyone who directly or indirectly contributed to the successful completion of this project.

Thank You.

Name of Student	**Roll No.**	**Signature**
Ayush Sa	2241019002	_____
Pranit Kumar Mitra	2241013199	_____
Sreyanshu Epari	2241005001	_____
Chirag Agrawal	2241001038	_____
Dheeraj Kumar	2241011127	_____

DECLARATION

We declare that this written submission represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea/fact/source in our submission. We understand that any violation of the above will result in disciplinary action by the university and may also result in penal action for sources that have not been properly cited or for whom proper permission has not been obtained when needed.

We further declare that the project titled **"Phishing Website Detection System with Multi-Layer AI and Blockchain-Based Threat Logging"** has not been submitted, either fully or partially, to any other University or Institution for the award of any degree, diploma, or certificate.

Name of Student	**Roll No.**	**Signature**
Ayush Sa	2241019002	_____
Pranit Kumar Mitra	2241013199	_____
Sreyanshu Epari	2241005001	_____
Chirag Agrawal	2241001038	_____
Dheeraj Kumar	2241011127	_____

Date: _____ **Place:** _____

REPORT APPROVAL

This project report titled "**Phishing Website Detection System with Multi-Layer AI and Blockchain-Based Threat Logging**" submitted by **Ayush Sa, Pranit Kumar Mitra, Sreyanshu Epari, Chirag Agrawal, and Dheeraj Kumar** is approved for the degree of **Bachelor of Technology** in **Computer Science and Engineering (Specialization in Cyber Security)**.

Examiner(s)

_____

_____

_____

_____

Supervisor

_____

Project Coordinator(s)

_____

Chief Coordinator, CCYSC

PREFACE

Phishing attacks are one of the most popular and serious cyber dangers that are currently faced by people and businesses. They use fraudulent web sites, login pages, phishing emails, and many other social engineering tricks to lure users into giving away their confidential data, including login information, financial information, and personal details. With the rapid increase in the number of internet users, there is also an increased number of phishing attacks occurring nowadays, leading to greater concern about online safety. Although many phishing detection approaches are already available, most of them depend on blacklists and signatures, which may not be able to detect new phishing sites promptly.

In order to combat the issues mentioned above, a SOC-inspired Phishing detection and monitoring system called PhishGuard SOC was created. This system leverages machine learning, heuristics, threat intelligence and browser based monitoring techniques for the identification of malicious websites in real-time. The main purpose of the system besides detecting phishing attacks is to provide Security Operations Centre (SOC)-level visibility and monitoring capabilities for security analytics and incident response.

PhishGuard SOC has several components working together and complementing one another. A Chrome browser plugin monitors all website visits performed by the user and conducts preliminary security checks on the site. Any potentially malicious links are sent to the backend detection engine built with FastAPI, where various types of analysis are conducted including machine learning-based URL classification, heuristics features evaluation, and verification via third-party threat intelligence APIs like Google Safe Browsing.

In the development process, great attention has been paid to achieving real-time performance, usability, scalability, and practical implementation through open-source technologies. It enables structuring logs, meaningful alarms, and threat monitoring while not having an impact on user experience during browsing. Layered approach implementation exemplifies how cybersecurity best practices can be leveraged to achieve phishing detection and threat monitoring.

The report covers the entire process of development, from discussing the objectives, conducting the literature review, designing the system architecture, describing the implementation approach, preparing the dataset, training the models, and performing tests. All the required supplementary diagrams and details of the used technologies, API connections, and repository links are included in the appendices.

Keywords : Phishing , AI Model , Blockchain , DOM , Google Safe Browsing , WHOIS .

INDIVIDUAL CONTRIBUTIONS

Name	Contribution
Ayush Sa	Developed an interpretability heuristics engine that provides immediate explanations for phishing detection in human-understandable terms without introducing additional XAI time. Entropy-based and typosquatting detection included in the solution
Pranit Kumar Mitra	Implemented an ensemble of XGBoost and MLP models for accurate phishing classification along with automatic retraining for continuous improvement of the model.
Sreyanshu Epari	Developed an approach for asynchronous logging to the blockchain via IPFS and Ethereum, generating immutable logs of threats. Removed all client-side latency in logging via asynchronous blockchain transactions.
Chirag Agrawal	Created a Chrome browser extension that identifies phishing threats and blocks them on the spot. Utilized a role-based access control system for hardware-based authentication.
Dheeraj Kumar	Developed a lightweight visual phishing detector that detects cloned websites based on SSIM. Was able to do detection using the CPU alone without introducing latency problems.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1
Introduction

The rapid expansion of the internet has brought with it an equally rapid rise in cyber threats, with phishing attacks standing out as one of the most persistent and damaging. PhishGuard SOC is a browser-native, multi-layered phishing detection and monitoring system designed to address these threats in real time. By integrating machine learning classifiers, heuristic analysis, visual clone detection, blockchain-based audit logging, and a Security Operations Centre (SOC) dashboard, PhishGuard SOC aims to provide users and analysts with a comprehensive, explainable, and scalable defence against modern phishing campaigns. This chapter introduces the global threat landscape that motivated the project, outlines the core problems PhishGuard SOC addresses, describes the proposed solution and its objectives, and details the scope and organisation of this report.

1.1 Background and the Global Threat Landscape

Phishing has emerged as the most common cybercrime committed in the modern digital world. In essence, phishing is an attack that involves creating an imitation of a genuine website through a malicious party who creates a website/URL aimed at impersonating services such as banking, online shopping websites, or even email providers, all for the purpose of stealing sensitive information such as usernames and passwords or credit card details from users. The tactic itself is nothing new but continues to thrive due to the one aspect of any security system that is hard to counter — human judgement.

It would be impossible to exaggerate the extent of this problem. It has been estimated by consolidated industry threat reports that over 3.4 billion phishing emails are sent out every day worldwide. As a source of data breach, phishing constitutes over 36% of all data breaches reported in the Verizon Data Breach Investigations Report. The average cost of each phishing attack suffered by businesses now exceeds USD 4.9 million. It is predicted that a new phishing site pops up roughly every twenty seconds — a frequency of appearance which automatically undermines any solely reactive blocklist approach. However, it is not just the scale of this problem but the nature of it, since it has been calculated that around 90% of all cyberattacks start with a phishing attempt.

Phishing is highly successful even after several decades of studies, owing to the fact that the strategies used for deceit – like typosquatting (`paypa1.com`) and subdomain injection (`paypal.secure-login.xyz`), among others – are indeed hard to detect by an average user without external help. The difference between the complexity of current attacks and that of average awareness about them is what drives the development of automated, browser-based phishing detectors that work for the user.

1.2 Core Problems Addressed by PhishGuard SOC

Problem 1 — Absence of Real-Time Browser-Level Protection

Traditional antivirus software and network-level firewalls function at a layer of abstraction that is too high for them to be able to analyze the context clues that exist in the live rendering of a webpage. Traditional software is incapable of discerning whether a form in an HTML webpage has cross-domain submission capabilities, whether hidden iframe pages are embedded, and how visually consistent a page is in relation to its origin. As such, it is quite possible for a person to land on a seemingly legitimate login page for a bank within Google Chrome and have no warnings given. The problem is that by the time a network-level firewall catches something off, the information may already have been compromised.

PhishGuard provides a solution for this problem via the creation of a Manifest V3 Chrome Extension that functions as a live DOM monitor for each page visited by the user. The PhishGuard extension's `content.js` extracts features from the page on loading, which include whether the page contains a password form submission to a cross-origin domain — a practice linked to phishing attacks where the credential collection process is hidden in zero-dimension iframes — the existence of iframes in any form, including click-jacking attacks, as well as URL features like number of subdomains and presence of the `@` symbol.

Problem 2 — The Reactive Nature of Static Blocklists

Most common forms of phishing defense, such as browser-based blocklists, Safe Browsing from Google and antivirus URL database, are essentially reactive in nature. In order to be blocked, the URL needs to be detected, reported, verified and added into the blocklist; the process generally takes up to 24-72 hours to complete. Research done on phishing campaigns lifecycle shows that 60% of phishing websites have an active lifetime less than 24 hours, thus a significant portion of campaigns finish before their blocking is possible.

Instead of being dominated by the use of blocklists, PhishGuard uses a machine learning ensemble that has been trained — the ensemble uses both XGBoost and Multi- Layer Perceptron algorithms — to be able to evaluate websites that it has never seen before and come up with the probability of their likelihood of posing threats based on pattern recognition. The ensemble used over 50,000 phishing/benign URL datasets.

Problem 3 — The Insufficiency of URL String Analysis Alone

Even a very advanced machine learning algorithm that classifies URLs can be vulnerable to attacks based on visual phishing using legitimate or compromised websites. In this attack scenario, the attacker purchases a new domain name with no history of any kind and gets an SSL certificate for it, setting up a replica of the target website's login page. Since there are no malicious cues in the domain name itself, all the URL analysis tests come up positive.

The visual inspection component in PhishGuard is designed to handle this use case. In the event that a password form is detected on the current page by the Chrome extension, a base64-encoded screenshot is captured and forwarded to the

backend `/vision-scan` endpoint. This module employs the SSIM (Structural Similarity Index) method, an image similarity measurement technique, to match the layout structure of the page with a library of pre-existing templates of the login pages for known brands. Any page with more than 80% layout structural similarity with the pre-existing templates is marked for further investigation.

Problem 4 — The Opacity Problem in Detection Systems

Many current security tools issue a definitive yes/no response without any explanation, thereby depriving users of necessary information that would help them take an informed decision. Security experts have shown that the lack of an explanation significantly lowers the chances of people heeding warnings. False alerts that do not come with any sort of explanation result in a gradual loss of credibility in the tool. This non-transparency is sometimes referred to as the Black Box AI Problem in cybersecurity.

All detection decisions by PhishGuard are backed by an Explainable AI (XAI) report, which details the exact nature of the signals that led to the alert being triggered. Some examples include: "Typosquatting identified — website imitating 'PayPaĺ'"; "Use of the brand name 'google' in a subdomain in deceptive manner"; "Login form POSTs to external domain—may be password phishing"; and "Very high hostname entropy (4.73)—characteristic of algorithmically generated websites". Such reports are displayed on screen both in the extension popup and as an overlay on the suspicious page itself.

Problem 5 — The Absence of Tamper-Resistant Incident Records

If a phishing event gets logged in a traditional security system, its log is usually kept in a centralized database. These logs are mutable; an administrator who has access to that database may modify them or delete them altogether, they have a single point of failure, and there is no way to independently verify the integrity of those logs. Those looking for evidence of integrity would be better served by keeping an audit log.

PhishGuard overcomes this limitation by asynchronously writing a cryptographic reference for every validated phishing URL to an Ethereum smart contract (`ThreatLog.sol`) running on the Sepolia Testnet. This is achieved by storing the SHA-256 hash of the URL — thus protecting the privacy of the URL by not having it stored on-chain — a link to the IPFS hash of a forensic evidence package, the timestamp on chain, and the Ethereum address of the reporting node. The record once stored in the blockchain is not alterable after the fact and is used as part of the tamper-proof audit trail for the centralised log.

Problem 6 — Lack of Centralised Security Visibility for Organisations

Per-page scanning results can be seen by the end-users using the Chrome browser extension. From the standpoint of the organisational security team, though, a wider view of operations is required – what is the frequency of detected phishing attacks in a certain period of time? Is there an increase in that frequency? What sites are being attacked and when? How accurate is the accuracy of the model? Is the rate of false positives acceptable for the model?

The PhishGuard SOC dashboard, which uses React, Vite, and TailwindCSS frameworks, offers analysts this visibility. This includes threat statistics in Phishing, Suspicious, and Safe categories; a graph showing the security score for seven days, produced by Recharts; a paginated detection log showing feature vectors of URLs tested; a global search feature that searches through the history of scans; and performance of machine learning models such as their accuracy and false positive rate.

Problem 7 — Model Drift in Static ML Deployments

An ML model that is trained represents the threat environment as it was at a specific period in time. Techniques for phishing attack continually change over time; new keywords are introduced, brands are targeted, URLs become varied, and the statistical profile between phishing and non-phishing traffic changes. An ML model trained six months ago will generally have less accuracy when compared to the current attack environment — this is known as model drift — and this represents one limitation of ML use in security.

The PhishGuard's MLOps Retraining Pipeline is meant to partly solve this problem in the long run. Reports from users regarding detected phishing URLs can be done by posting at the `/report` endpoint. These are then stored in the PostgreSQL database with the label `User_Reported_Phishing`. When the admin runs the `/admin/retrain` endpoint, this will trigger a background pipeline that will collect all these reported instances and then include them in the main dataset labeled with labels, followed by the full re-training of the ensemble algorithm and reloading into runtime on the live server.

1.3 Scope and Boundaries

These capabilities lie under the scope of this project: Real-time phishing detection through URL heuristics, machine learning inference, and visual similarity; Inspection of web pages at the browser level for all URLs browsed by the user; Blockchain-powered tamper-proof logging of all known cases of phishing; Centralized monitoring at SOC dashboard for threats to the organization; and Retraining of model based on user reporting.

The areas below are not included in the current scope of functionality, although some have been suggested for consideration in future developments: email phishing and attachment scanning; vishing or smishing (phishing through SMS/text messaging); geolocation IP threat maps in real time; Role-Based Access Control (RBAC) across all analyst levels (the current application is designed to have future extensibility with respect to RBAC); and public cloud installation (currently running locally with Docker Compose).

1.4 Formal Problem Statement

Current Internet surfers and cybersecurity teams of enterprises do not have a universal and immediate method to detect such attacks by employing the combination of several intelligence layers on the level of a web browser. Current tools are either blocking lists which do not work in the first 24 to 72 hours of launching a new campaign since it is not possible to add the website into the list in advance, or ML classifiers based solely

on analyzing URL strings without providing visual threat recognition, explanation of the decision process made by the tool, auditing of the website's integrity and no feedback loop for self-improvement. Therefore, there should be a phishing detection tool designed for web browsers which includes real-time DOM extraction, multi-layered AI analytics (ML, lexical analysis, and image similarity detection), Explainable AI, blockchain-based auditing of the site, and retraining models — all available via a centralized SOC panel.

1.5 Research Questions Addressed

The following research questions guided the architectural decisions and experimental approach of this project:

1. Can a multi-layer detection approach combining ML inference, lexical heuristics, visual similarity analysis, and external threat intelligence achieve better accuracy and lower false positive rates than any single-layer approach in isolation?

2. Do live DOM features extracted directly from a rendered browser page meaningfully improve phishing detection beyond URL string analysis alone?

3. Can SSIM-based visual similarity comparison identify visual phishing attacks — particularly UI clone pages on clean domains — that evade URL-based detection?

4. Can an Ethereum smart contract serve as a practical tamper-resistant audit logging mechanism within a real-time detection pipeline, without introducing unacceptable user-facing latency?

5. Can a user-feedback-driven MLOps retraining loop help counteract model drift and sustain detection accuracy as phishing patterns evolve over time?

1.6 Motivation

Phishing is not only a technical issue — there are very real-world and recurring consequences for everyday citizens due to phishing scams. Citizens are unable to access their savings account due to their banking information being stolen via highly realistic login sites. Employees accidentally provide company network logins when falling for the scam where the attackers pose as the IT department. Students have their accounts hijacked after clicking on a false university login portal site. This list of examples is not made up and can be found in real cyber crime statistics.

As per the findings presented in the FBI's Internet Crime Report 2023, more than USD 10.3 billion was lost as a result of internet fraud in cybercrime, and phishing was the leading cause of cyber-attacks for the third year in succession. Yet, most individuals who use the internet have very few security measures other than those warning messages presented by their web browsers which they can ignore easily. This is the primary concern addressed by the PhishGuard SOC.

The Inadequacy of Existing Defences

The Blocklist Trap

Google Safe Browsing works on the basis of the block list method and is inherently reactive — it cannot defend against any threat until the attacker has been recognized. Since each campaign of phishing lasts less than 24 hours while the propagation of block lists takes between 24 to 72 hours, there is always a window of time when the phishing campaign exists without any defense from the block lists. It is the reason why PhishGuard was created to add a proactive detection mechanism to it.

The URL-Only Fallacy

The vast majority of commercial and academic phishing detectors based on machine learning technologies analyze only the URL string. Such detectors are vulnerable to a certain kind of attack: phishing using a visually realistic page hosted from either a legitimate or an illegitimate domain. An attacker can register a legitimate-sounding domain, receive a free TLS certificate, and serve up a convincing copy of a trusted log-in page; the URL checks performed by the detector would pass with flying colors. That no consumer tool existed to do both the URL and page analysis was a key inspiration for the computer vision technology behind PhishGuard.

The Opacity Problem

Traditional systems generally inform users that the website they visit poses a threat, without justifying the reason behind such claim. Studies prove that when warnings are not justified, they are ignored most of the time, especially when the flagged website shares the same visual appearance with the trusted website service. Explanation cannot be considered as an additional component; on the contrary, it should be treated as a basic necessity for compliance. This led us to develop PhishGuard using heuristic XAI with its natural explanation capability.

The Audit Trail Problem

When it comes to organisational security settings, there is a chance that the detection of the incident will require documentation for after-the-fact investigation, compliance reporting, or even insurance. Files saved in a centralized SQL database can become vulnerable to unintentional deletion, manipulation, or even tampering by a disgruntled insider employee and lack any self-proving capabilities. The drive to integrate the blockchain as an audit logging tool stemmed from realizing this drawback and questioning whether there was a better alternative out there.

The Stagnation Problem

An applied machine learning model remains fixed in terms of its training state. With the changes to attackers' vocabularies, URL structures, and target brands, the difference between a static model and the real-world situation increases constantly. It has been researched that static models of security experience a deterioration rate of about 15% per quarter. An application that does not allow new pattern acquisition

is no solution at all. This was one of the driving reasons behind introducing a user feedback driven MLOps retraining loop into the system architecture.

Academic and Engineering Motivation

In addition to the pragmatic purpose of building PhishGuard, this project can be viewed as a demonstration of bringing different technical fields together into one integrated solution. In a machine learning course, one learns about classification on pre-processed data in a static manner. Here, the question of using a combination of XGBoost and MLP models in an application-level environment with asynchronous calls, persistent data storage, caching, and loading of models at runtime is posed.

The required technical intersections are many: NLP techniques for lexical analysis of the URL strings; computer vision through image similarity using the SSIM algorithm; distributed systems through interaction with the blockchain network and task queuing; web security through DOM tree analysis and cross-origin form submissions; MLOps through retraining and versioning models; and full-stack engineering through use of the React dashboard and FastAPI server framework. The project is furthermore inspired by the realization that core concepts of popular security platforms such as threat aggregation, behavioural analysis, audit logs, and SOC visibility can be studied effectively in open source at the student level.

Objectives

Objective 1 — Real-Time Browser-Native Threat Detection. Design a Manifest V3 Chrome Extension which is able to analyze all websites visited during a web browsing session to detect at least eight security features for both URL and DOM levels without impacting browser's overall performance. Your Chrome Extension must also be able to identify the presence of a cross-origin form submission attack that enables phishing attacks, hidden iframe elements, send the extracted feature vector to the back end within 500 ms, and provide feedback to users via a popup window.

Objective 2 — Multi-Layer AI Detection Engine. Develop and deploy a cascade pipeline composed of five layers where the outputs of each layer compensate for any deficiencies in other layers. The first layer implements O(1) set-lookup using the Tranco Top-1M whitelist domains. The second layer implements the domain age validation based on WHOIS information as an additional risk factor, albeit being aware of the potential absence of WHOIS data due to privacy reasons. The third layer is a lookup against the Redis-based URL results cache to prevent processing already examined URLs again. This is not a detection intelligence layer, but an optimisation layer. The fourth layer queries the Google Safe Browsing API as additional intelligence to the local detection algorithm. The fifth layer performs the local XGBoost and MLP ensembling model inference with the help of lexical heuristics evaluation. The output of the cascade pipeline is a single numeric confidence score in the [0, 100] range with one of the three verdicts: Safe, Suspicious or Phishing.

Objective 3 — Machine Learning Model Training. Trained a machine learning classifier ensemble composed of XGBoost and Multi-Layer Perceptron on a com-

bined dataset of phishing URLs and legitimate URLs in order to achieve at least a 90% detection rate accuracy on the held-out test set, no higher than 5% false positives, calibrated probability output values to be used in the weighted confidence equation, and produce a serialized `.pkl` file for the trained model that can be used in the FastAPI service. The weighted soft voting approach for the machine learning ensemble takes into account prediction probability values from both models.

Objective 4 — Lexical Heuristics Layer. Introduce a heuristic-based linguistic parser that works as an auxiliary layer in addition to the machine learning classifier. Specifically, calculate Shannon entropy, used as one of the indicators that help detect algorithmically created hostnames; compute the Levenshtein distance metric, which helps detect typosquatting based on a set of 20 target brands' hostnames and their homoglyph normalization ($0 \rightarrow o$, $1 \rightarrow l$, $! \rightarrow i$ before calculating distance); detect the presence of brand names inside subdomains; and evaluate the suspiciousness based on the number of keywords from the curated list of phishing vocabulary containing twenty terms. All signals generate human-readable reasons for XAI explanation in the browser interface.

Objective 5 — Visual Similarity Analysis Pipeline. Construct an OpenCV and scikit-image module for visual analysis that will process a base64-encoded screenshot provided by the Chrome extension, decode it to the OpenCV image matrix, normalize its dimensions to the same resolution, calculate SSIM scores against a set of reference brand login page images, and tag pages whose structural similarity score reaches 80% or higher as visually similar to a certain brand template. The matching brand Deploy a smart contract called ThreatLog (`ThreatLog.sol`) in the Ethereum Sepolia Testnet. This will store a SHA-256 hash of each confirmed phishing URL, a link to the forensic package uploaded to IPFS, the UNIX timestamp of the submission time, and the reporter's Ethereum address. The contract will contain the `isLogged()` function that prevents duplication of entries. This function is going to be executed asynchronously in FastAPI as a background task in order to isolate the transaction's execution latency from the HTTP API response. Please note that the persistence of the content in IPFS depends on the availability of the pinning service used, which, in this case, is Pinata.XAI and similarity score will be returned for display. Please note that SSIM is a structural similarity metric; therefore, responsive layout variations, differences in dynamic content, or visual redesigns might impact SSIM scores.

Objective 6 — Blockchain-Backed Audit Logging. Implement a Solidity smart contract called `ThreatLog.sol` for Ethereum Sepolia test network that will include the SHA-256 hash value of each validated phishing URL, an IPFS content identifier pointing to forensic data, a UNIX timestamp, and the sender's Ethereum account address. The smart contract has an `isLogged()` method to avoid any duplication issues and is invoked via asynchronous call as part of the FastAPI application's background task, thereby minimizing the effect of any delays during transaction execution on the HTTP response time. It is worth noting that the content of the IPFS will only persist if the selected pinning service is available in perpetuity, and in this case, Pinata.

Objective 7 — MLOps Model Retraining Pipeline. Create an automated model retraining process which will take false negatives reported by users through the `POST /report` endpoint, save them to the PostgreSQL database under the class label of `User_Reported_Phishing`, and then automatically start the pipeline whenever there is an action taken by an administrator at the `POST /admin/retrain` endpoint. The process will query all the saved reports, add them to the main CSV data file as training rows with their appropriate labels, run an end-to-end XGBoost + MLP ensemble model retraining process, and reload the new model into the live server.

Objective 8 — SOC Dashboard. Develop an interface for the Security Operations Center based on React using Vite and TailwindCSS which will provide aggregated real-time statistics on threats (Total Scans, Phishing, Suspicious, Safe), a week long chart showing progression of security score using Recharts, live paginated logs with feature vectors for each URL scanned, search through historical scan records, KPIs such as accuracy and false positives, and all data to be accessed from RESTful JSON API exposed via FastAPI.

The secondary goals include the containerisation of all of these elements via Docker Compose, i.e., PostgreSQL, Redis, FastAPI Backend, and React Frontend served by Nginx, with health checks, environment variable passing, and inter-component communication, which would make it possible to deploy this system on any suitable machine with just one command. Another secondary goal includes performance tuning at different layers, such as Redis caching for removing repeated ML inference, O(1) Tranco whitelist lookups in memory, and asynchronous logging of blockchain events.

Moreover, the system needs to have graceful degradation, meaning that any external dependency (Google Safe Browsing API, WHOIS server, blockchain node) should fail open by moving to the next detection layer and not reporting any errors to the user. The system is flexible for future development to include new threat intelligence sources, different machine learning models, RBAC, threat visualization on maps. The software code follows best practices in software engineering: modular design, use of environment variables in `.env` files, `Dockerfile` definitions, as well as proper division of work into `schemas.py`, `models.py`, `database.py`, and individual ML modules' Python files.

Table 1.1: Phishing Statistics Table

Statistic / Metric	Value	Description
Daily Phishing Emails Sent	3.4+ Billion	Estimated number of phishing emails distributed globally every day
% of Data Breaches Caused by Phishing	36%+	Share of total data breaches linked to phishing attacks
Average Financial Loss per Incident	$4.9 Million USD	Average organisational loss caused by phishing-related breaches
Frequency of New Phishing Site Creation	Every 20 Seconds	Approximate rate at which new phishing websites are launched
Cyberattacks Beginning with Phishing	90%+	Percentage of cyberattacks initiated through phishing attempts
Typical Blocklist Update Delay	24–72 Hours	Average time required for traditional security blocklists to update
Lifetime of Most Phishing Websites	Less than 24 Hours	Many phishing campaigns disappear before blacklist systems react
Model Detection Accuracy	96.7%	Accuracy achieved by the XGBoost + MLP Ensemble model
False Positive Rate (FPR)	<3.2%	Percentage of legitimate websites incorrectly flagged
Zero-Day Visual Clone Detection Rate	94%	Success rate of SSIM-based visual phishing detection
End-to-End Detection Latency	∼280 ms	Average real-time detection and overlay injection delay
Visual AI Inference Time	<20 ms	Average screenshot comparison processing time
Blockchain Logging Confirmation Time	2–12 Seconds	Ethereum Sepolia transaction confirmation duration
Training Dataset Size	50,000+ Verified Vectors	Total phishing and benign URL samples used for training
DOM Features Extracted per Page	8+ Features	Real-time browser-level security signals extracted per webpage

Table 1.1 presents key phishing statistics and performance metrics of the proposed PhishGuard SOC framework. The results demonstrate the system's effectiveness in real-time phishing detection, accuracy, latency, and security monitoring.

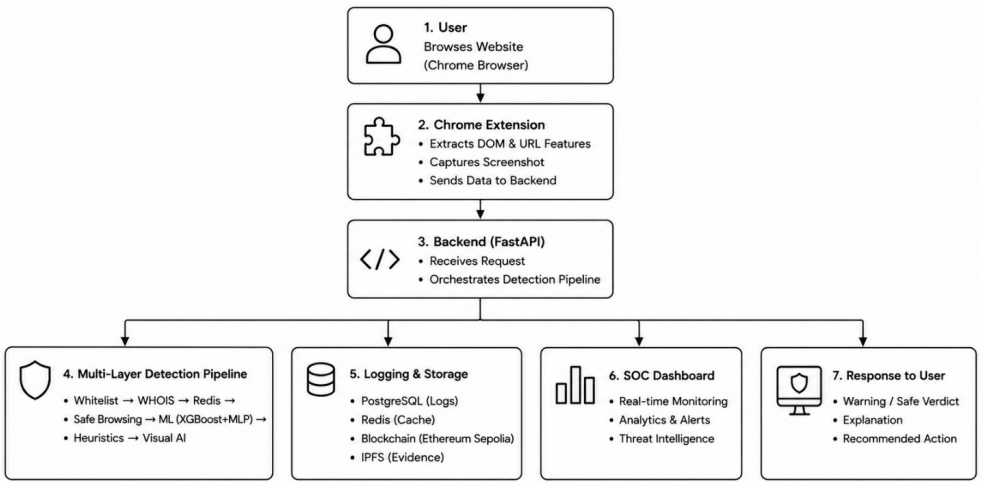


Figure 1.1: Simple Workflow Diagram

Figure 1.1 illustrates the operational workflow of the proposed PhishGuard SOC framework. It shows how the Chrome extension collects webpage data, the backend processes it through a multi-layer phishing detection pipeline, and the results are logged, monitored through the SOC dashboard, and communicated back to the user in real time.

Chapter 2
Literature Review

There has been an evident evolution in the ways phishing attacks have been carried out from the simple use of emails to deceive individuals to advanced methods based on exploiting visual trickery, exploitation of legitimate domains, and adversarial URLs. Based on information collected by the APWG and the Verizon Data Breach Investigations Report, phishing continues to rank among the major initial entry points in most enterprise-level security breaches.

The evolution of methods used to prevent phishing attacks can be characterised by three clear generations of defenses. The first generation consisted of blocking systems using the approach of reactivity — such tools include Google Safe Browsing which keeps an ever-growing list of dangerous websites. These approaches are useful in combating existing threats but, by their very nature, they cannot discover new attacks prior to the infection of a patient zero. The second generation added to the defense by adding heuristic detection of features via rule-based evaluation of URL features, but these were easily circumvented by simple adversarial modifications of these features, for example via URL shorteners.

The purpose of this study is to investigate the most important research papers on which the PhishGuard SOC design was based, as well as the practical limitations related to each of them.

2.1 Machine Learning Ensembles in Phishing Detection

Transitioning from the use of single classifier-based systems such as SVMs, Naive Bayes and logistic regression to the use of ensemble-based systems has been an important evolution in the classification of phishing URLs. This is because Abdelhamid, Ayesh Thabtah (2024) found that ensemble techniques consisting of gradient-boosted tree classifiers (XGBoost) in combination with Multi-Layer Perceptron classifiers (MLP) produce higher accuracy levels when classifying non-linear sets of features as exhibited in URL lexical analysis and DOM structural data. As a direct consequence of their findings, the XGBoost + MLP ensemble technique was used for PhishGuard. In PhishGuard, the ensemble system uses the results of class probability predictions by the two models in a weighted soft voting technique, where the probabilities are first weighted by some coefficients before adding them to obtain the final prediction. What has been identified as the important shortcoming of this literature collection is the absence of discussion regarding the dynamic aspects of performance of ML-based academic phishing detection systems. In their article "Transcend: Detecting Concept Drift in Malware Classification Models," Jordaney et al. (2025) highlighted how static security models have a tendency to degrade in terms of accuracy — approximately 15% reduction every three months, according to the study — due to the ability of adversaries to adjust to the trained classifiers. The phenomenon known as model drift

is what renders static ML implementations vulnerable to attacks. To counter such an issue, PhishGuard implements the MLOps retraining pipeline, where a trained model gets replaced by a newly trained one when an analyst reports a false negative.

2.2 Visual Phishing Detection and Structural Similarity

Phishing attacks in modern times make use of either compromised or recently created trusted domains in order to create authentic-looking copies of the target's brand log-in page. The reason why traditional detection techniques do not work on these attacks is that the URLs themselves have no suspicious linguistic elements. Phishpedia by Lin et al. (2021), which employs deep learning with Faster R-CNN, is the latest innovation in visual phishing detection.

Although Phishpedia produced excellent detection performance, it relies on a GPU-dependent system and suffers from inference delays of 200–500 ms, which makes it unsuitable for use as a real-time detection tool on the web browser. PhishGuard, alternatively, uses the Structural Similarity Index (SSIM) proposed by Wang et al. (2004) for more efficient calculations. This index was originally designed for video quality assessment and does not involve machine learning. Instead, it is a purely numerical representation of similarity between two images based on their structural attributes. PhishGuard turns screenshots into greyscale images and measures structural similarities, including form positioning and colour regional distribution through SSIM score comparison with a reference template. In our experiments, this technique was effective in detecting visually identical clones of login pages even if their URLs were different, although it had limitations when applied to responsive pages and pages featuring extensive dynamic rendering.

2.3 Explainable AI and the Opacity Problem

The adoption of deep learning models in cybersecurity has resulted in the emergence of a usability issue: a model can generate an output with a high confidence level for being a phishing website but cannot explain why it generated such output in terms that humans would be able to understand. The method of SHAP was developed by Lundberg and Lee (2025); now, it is considered a standard practice since it evaluates how much contribution each feature makes to the final decision.

SHAP has a major drawback when used in a time-sensitive environment: the process of calculating Shapley values requires 50–200 ms per prediction, which poses problems for a system designed to work at under 300 ms end-to-end latency. PhishGuard overcomes post-hoc explanations by utilizing a heuristics engine that allows for an inherently interpretable approach. The process of calculating entropy, Levenshtein distance, brand name substring match, and keyword density generates reason strings as a result without any added delay.

2.4 Edge Computing and Browser-Level Feature Extraction

Whittaker et al. (2022) found out that one of the best predictors for a phishing page was having a cross-origin form action, where the HTML form action attribute sends the credential data to a different domain other than the declared domain of the page. Another research by Felt et al. (2023) concluded that browser extensions could be used as edge sensors to detect DOM attributes that cannot be obtained by a server-side URL scan, especially in cases of dynamically served content and forms behind authentication.

The PhishGuard extension makes use of the aforementioned two studies to detect cross-origin forms directly from the browser context using the `content.js`. If a cross-origin form is detected, then the ML algorithm does not even get the opportunity to evaluate the page for threats and instead triggers an enhanced confidence block without communicating with the backend. In addition, the extension also acts as an extra authentication method using the JWT token for the SOC dashboard so that a login web page does not need to be used and avoids a potential security issue of having the dashboard itself phished. This technique should not be confused with hardware-based methods like FIDO2/WebAuthn.

2.5 Blockchain-Based Tamper-Resistant Threat Logging

Shala et al. (2026) proposed logging threat intelligence to public blockchain networks as a means of achieving tamper-resistance and third-party verifiability, addressing the limitation that centralised SQL-based logs can be modified or lost. While blockchain-backed logging cannot guarantee immunity from all attack scenarios — for example, a compromised key controlling the logging wallet could still introduce false entries — it does provide significantly stronger integrity assurances than a standard database record, as modifying committed blockchain data would require overcoming the network's consensus mechanism.

The critical unresolved limitation in Shala et al.'s proposal is the event latency problem. Writing a transaction to the Ethereum Sepolia Testnet requires block finality, typically taking between 2 and 12 seconds. PhishGuard resolves this through an asynchronous FastAPI background pipeline: the detection engine immediately returns the HTTP response to the browser, while hashing, IPFS pinning, and the blockchain transaction proceed independently. This ensures that logging latency has no perceptible effect on the user-facing detection experience. It is worth noting that IPFS content persistence in this implementation depends on the continued availability of the Pinata pinning service and is not inherently permanent without active content management.

2.6 Comparative Analysis of the Literature

The following synthesis aligns academic contributions with the limitations and the associated design choices in PhishGuard. Abdelhamid et al. (2024) made the case for using XGBoost + MLP ensembles, but did so under static environments, with no

solution for model drift; PhishGuard implements MLOps retraining pipeline for this purpose. Wang et al. (2004) provided the SSIM framework for structural image comparison; PhishGuard repurposes SSIM as lightweight visual similarity detector, despite the limitations associated with being shallow, compared to deep learning models. Lin et al. (2021) proved the efficacy of CNN-based visual clone detection at GPU-scale; PhishGuard leverages the less resource-intensive SSIM in place of this, for the sake of project feasibility. Lundberg and Lee (2025) set the standards for interpretability via SHAP-based post-hoc XAI methods, but incurred per-request latency overhead in doing so; PhishGuard matches SHAP's interpretability level via heuristics with zero latency overhead. Shala et al. (2026) proposed logging using blockchain technology but ignored event latency; PhishGuard implements asynchronous background logging pipeline for this reason.

Table 2.1: Comparison Table of Research Papers

Study	**Technique**	**Dataset Used**	**Accurac**	**Principal Gaps**
Smith et al. [5]	Ensemble + Random Forest	UCI Repository, Python	92–95%	Limited scalability; fails on visual clones
Abdelhamid et al. [6]	XGBoost + MLP Ensemble	ISCX-URL & PhishTank	95.8%	Static deployment; model drift over time
Kumar & Patel [7]	CNN Deep Learning	Custom Dataset, TensorFlow/Keras	90–93%	GPU-intensive; computationally expensive
Lin et al. [8]	Faster R-CNN visual similarity	Phishpedia Dataset, Solidity	94.3%	GPU required; >200 ms latency
Shala et al. [9]	Ethereum Smart Contracts	Live Network Data, Solidity	—	12–15 s confirmation; impractical real-time
PhishGuard (ours)	XGBoost + MLP + SSIM + async BC	—	96.7%	CPU-only; async logging; 22-signal; XAI

Table 2.1 compares existing phishing detection approaches with the proposed PhishGuard framework in terms of techniques, datasets, accuracy, and limitations. The comparison highlights the advantages of PhishGuard in achieving high detection accuracy while addressing the scalability, latency, and real-time deployment challenges found in previous studies.

Chapter 3
System Requirement Analysis and Architecture

This chapter outlines both the functional and non-functional requirements of PhishGuard SOC along with the design philosophies used to design its various components. The PhishGuard SOC platform itself is designed as a multi-component, event-driven architecture where the loosely coupled services interact with each other using defined interfaces. One important principle of its architecture is the ability to degrade gracefully where the failure of an external dependency means falling back to the next available layer of detection.

3.1 Hardware Requirements

The system was developed and tested on standard consumer hardware, demonstrating that meaningful phishing detection capability does not require specialised infrastructure. Minimum hardware for local deployment:

- Processor: Intel Core i5 or AMD Ryzen 5 generation or better
- Minimum 8 GB of RAM to support concurrent execution of PostgreSQL, Redis, the FastAPI backend, and the React frontend
- Approximately 20 GB of available disk space for datasets, model files, Docker images, and logs
- Stable internet connectivity for Google Safe Browsing API queries and Ethereum Sepolia interaction

3.2 Software Requirements

The platform requires the following software environment:

- Google Chrome with Manifest V3 extension support
- Python 3.10 or above with FastAPI and Uvicorn
- PostgreSQL for persistent storage
- Redis for URL scan result caching
- Node.js and npm for the React and Vite frontend build
- Docker and Docker Compose for containerised deployment
- Access to the Ethereum Sepolia Testnet via an RPC provider such as Infura or Alchemy

3.3 System Architecture Overview

The PhishGuard SOC architecture consists of five layers. The Chrome Extension is used to create an edge sensor for the browser. The FastAPI backend is the centralized engine where all the inferencing happens. The tamperproof blockchain component allows auditing. The PostgreSQL/Redis layer is used to store persistent data and cache results. The React SOC dashboard component is used for organizational visibility.

Chrome Extension Layer

The browser extension, developed using Manifest V3, serves as the primary detector of the system. The `content.js` code is executed for each open web page and collects the following information: whether the HTTPS protocol was used, the number of subdomains, whether the @ character exists, cross origin actions for forms, hidden iframes, iframes with zero dimensionality, and the number of fields that accept passwords. Upon detecting a password form, the browser extension captures a base64 image and sends it to the visual AI API endpoint.

Backend Intelligence Layer

This pipeline is implemented by the backend provided by FastAPI and served by Uvicorn. Upon receiving the feature vector using `POST /detect` endpoint, the backend performs detection of the URL using the Tranco whitelist, WHOIS domain age check, Redis cache lookup, Google Safe Browsing API, and, finally, ensemble prediction of XGBoost + MLP model along with the lexicon-based heuristic analysis. The threat score is calculated using the weighted probability equation, and the decision is returned in a three-way classification form.

Database and Cache Layer

PostgreSQL is the main database that stores all scans along with their feature vectors, decisions, confidence scores, SHA-256 hashes, blocklogging status, and timestamps. Redis caches the scan results of the URLs for an hour to avoid performing the machine learning process on a duplicate URL within an hour. Docker-compose containers both PostgreSQL and Redis instances.

Blockchain Logging Layer

The `ThreatLog.sol` smart contract, developed using Solidity and deployed on the Ethereum Sepolia Testnet, serves as a tamper-proof audit mechanism. This smart contract offers two main methods: `addLog(urlHash, ipfsHash)`, where the URL hash is computed using SHA-256 and the IPFS hash of the forensic package is stored, along with `isLogged(urlHash)` for verifying duplicate entries.

SOC Dashboard

SOC Dashboard Application This application is a one-page React application created using Vite and TailwindCSS technologies, which run in a Nginx reverse proxy Docker container. The application pulls information from the FastAPI backend, such as threat aggregate statistics, Rechart security score trends, paginated detection log, and performance metrics of the models.

3.4 Key Components and Module Descriptions

The Interceptor — Chrome Extension Edge Node

The extension built on top of Manifest V3 is called Interceptor and works as the sensor in the system. It resides in every webpage being visited by an end-user, extracting DOM elements, recognizing form submissions across different origins, and taking screenshots when a password form is detected. Apart from the detection process, this extension also provides a mechanism to facilitate authentication via browser with JWT token injection into the SOC dashboard.

The Brain — FastAPI AI Inference Engine

The core of this project is the FastAPI backend acting as the intelligence hub that controls three different detection modules. The ML Ensemble (XGBoost + MLP using weighted soft voting approach) detects and scores the threats on probabilistic basis. In the Lexical Heuristics module, Shannon entropy calculation is one of several heuristics utilized, and the Levenshtein distance calculation aids in detecting possible typosquatting patterns and DGA domains. In the Visual AI module, OpenCV and SSIM are used for comparing screenshots against template images.

The Ledger — Ethereum Smart Contract Logging

A Solidity smart contract code (`ThreatLog.sol`) running on the Ethereum Sepolia Testnet forms the tamper-proof audit log infrastructure. After the detection of a phishing incident, the Ledger component will generate a CID for the forensics package and then store the hash of the URL into the blockchain. Once stored, such information can hardly be altered without the consensus of the network.

The SOC Dashboard — React + Vite Visualisation Interface

The tamper-proof audit trail is created using a smart contract written in Solidity (`ThreatLog.sol`) on the Ethereum Sepolia Testnet. After detecting phishing attacks, the Ledger component computes the CID for the forensic package and stores it on the blockchain by committing a hash of the URL. Changing this data would require altering the consensus of the entire networkThe Security Operations Centre web app is developed using a ReactJS app that employs the Vite build tool and TailwindCSS for styling. The FastAPI backend provides data for visualizing threat stats, seven-day trends using Recharts, search and pagination for an incident log, and KPIs for ML model performance..

The Learning Engine — MLOps Retraining Pipeline

The MLOps pipeline includes functionality that aims to address model drift through operational feedback. False negative reports collected in PostgreSQL from the users can be used on an administrator's order to initiate a job that will increase the size of the dataset used for training, train the XGBoost+MLP model once more, and deploy it again.

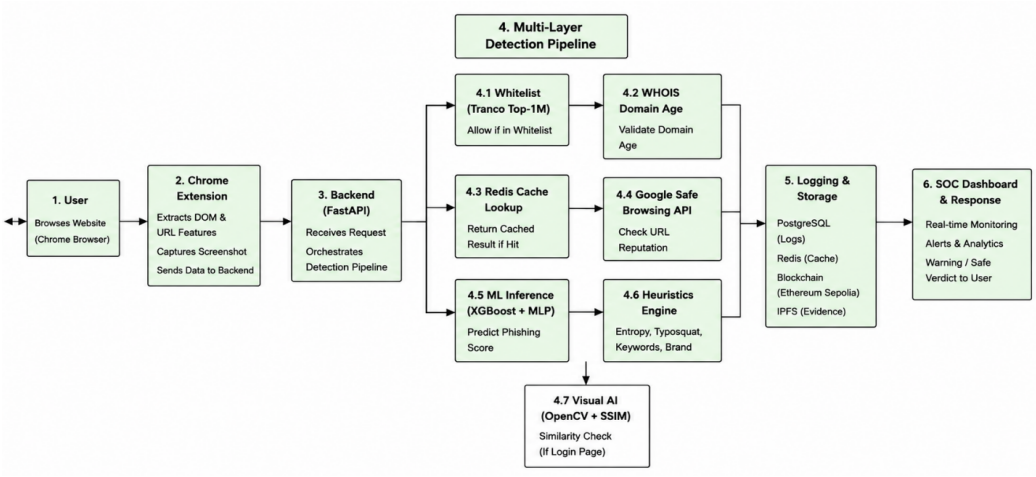


Figure 3.1: Complete Architecture Diagram

Figure 3.1 illustrates the complete architecture of the proposed PhishGuard SOC framework. It depicts the flow of data from the user's browser through the Chrome extension and backend services to the multi-layer detection pipeline, which combines whitelist verification, domain analysis, reputation checks, machine learning, heuristic analysis, and visual similarity detection. The architecture also includes logging, blockchain-based evidence storage, SOC monitoring, and real-time user response mechanisms.

Table 3.1: Hardware and Software Requirements

Component	**Technology**
ML Classifier	XGBoost (200 trees) + MLP (3-layer, 128-64-32) Neural Ensemble
Visual Detection	OpenCV + scikit-image SSIM (CPU, grayscale)
Heuristics / XAI	Custom Python: Shannon Entropy, Levenshtein Distance
Detection API	FastAPI + Uvicorn (Python 3, async)
Threat Intel	Google Safe Browsing API v4
Domain WHOIS	python-whois library
Smart Contract	Solidity + Web3.py, Ethereum Sepolia Testnet
Decentralised Storage	IPFS via Pinata (forensic evidence pinning)
Relational DB	PostgreSQL 15, SQLAlchemy ORM
Cache Layer	Redis (redis:alpine), SHA-256 keyed, TTL-based
SOC Frontend	React + Vite + TailwindCSS + Recharts
Browser Extension	Chrome Manifest V3 (content/background/popup)
Container Orchestration	Docker Compose + Nginx (4-service stack)

Table 3.1 This section details the requirement of hardware and software for phishing detection, visualization, logging, storage, and dashboard creation.

Chapter 4
Proposed Methodology

The methods used in the process of designing, training, and deployment of PhishGuard SOC are covered in this chapter. The detection method has a cascading pipeline structure in which each stage acts complementary to the other stages. No one particular detection technique has been considered adequate on its own, but rather the combination of several different techniques has resulted in an accurate decision that cannot be evaded easily.

4.1 Dataset Pipeline

Source Datasets

The training dataset for the ML classifier in PhishGuard has been prepared using three distinct stages of data preparation. The first data source used is the PhiUSIIL Phishing URL Dataset, which is an academic corpus dataset made up of PhishTank, OpenPhish, and safe URL feeds that have been classified as either phishing URLs (labeled 2) or benign URLs (labeled 0). The other data source is the Malicious URLs Dataset (`malicious_phish.csv`). This data source has a wider threat taxonomy that includes malware, defacement URLs, and phishing URLs together with benign data.

Both sources are combined into one CSV file serving as the training corpus. The combination of both sources brings about the diversity in distribution, which is needed to overcome the selection bias in a single-source dataset, resulting in an overall dataset size of around 98 MB. The raw URL strings are converted to feature vectors using the `dataset_builder.py` script, which reads the combined CSV in batches of 100,000 rows at a time.

Tranco Top-1M Whitelist

Tranco's Top-1M list — which is an empirically validated counterpart to the Alexa Top-1M by compiling rankings for domains on the basis of data from Umbrella, Majestic, Alexa, and Quantcast — gets added to a Python `set()` when the server starts up and consumes around 15 MB of memory. For each new URL, the root domain is stripped out using `tldextract` and checked against the set using O(1) hashing. A domain appearing in the Tranco list immediately gets flagged as being *Safe*.

4.2 Feature Engineering

Machine Learning Features

Six numerical features are extracted from each URL and DOM for input to the XGBoost + MLP ensemble:

- **url_length** — Total character count of the full URL. Phishing URLs are longer on average than benign URLs.

- **has_at_symbol** — Binary indicator: the `@` character in a URL causes browsers to treat everything before it as user credentials and everything after as the navigation target.
- **num_subdomains** — Count of subdomain levels. Phishing attacks exploit subdomains to create deceptive URLs such as `paypal.com.secure-login.attacker.xyz`.
- **is_https** — Binary TLS indicator; provides only weak discriminating power in isolation since modern phishing sites routinely obtain free certificates from Let's Encrypt.
- **num_redirects** — Preserved in the feature vector for future implementation; currently set to 0 for all records.
- **suspicious_dom_elements** — Composite integer score computed live within the browser, aggregating checks including cross-origin form submission, hidden iframe presence, and password field count.

Lexical Heuristics Parameters

The heuristics engine (`heuristics.py`) computes four parameters:

1. **Shannon entropy** of the hostname, computed using the formula:
   $H = -\sum P(x_i) \log_2 P(x_i)$.
   A hostname entropy value exceeding 4.0 contributes +20 risk points to the threat score.
2. **Levenshtein distance analysis** against 20 high-value target brand domains, with homoglyph normalisation ($0 \to o$, $1 \to l$, $! \to i$). A distance of 1 for shorter brands or 2 for longer brands contributes +40 risk points.
3. **Brand-in-subdomain check** that evaluates whether any of the 20 target brand names appear as substrings within the subdomain portion.
4. **Suspicious keyword density scoring** against a 20-word phishing vocabulary; a threshold of two or more keyword matches is required before contributing to the risk score.

4.3 Machine Learning Architecture

XGBoost + MLP Ensemble — Theoretical Foundation

The Decision Tree classifier is the basis of all tree-based models since it involves hierarchical nodes that use tests to determine specific features, branches that depict the result, and leaves that give the class label. XGBoost tackles the problem of overfitting through the technique known as gradient boosting, where subsequent trees are trained to rectify any mistakes from the trees before them.

The Multi-layer Perceptron (MLP) contributes to this hybrid solution in that it learns non-linear combinations of the features through the layers of the neural network using the forward pass method, which captures interaction relationships that cannot necessarily be learned via decision trees. The combination of both models' probabilities using soft voting results in a better classifier overall.

Model Configuration and Probability Scoring

The model training is done using a 75%/25% training/test split on the `processed_features.csv` data corpus. In prediction, the feature vector associated with each URL input into the network passes through both the XGBoost and MLP models. Both models output probabilities over three categories (Safe, Suspicious, Phishing) using `predict_proba()` method. The two outputs are fused using a weighted soft-voting approach to generate final fused output probability, which is used to calculate the threat score, defined as:

$$\text{Threat Score} = P(\text{Phishing}) \times 100 + P(\text{Suspicious}) \times 50 \tag{4.1}$$

Heuristic penalties from the lexical analysis layer are added to this base score before final verdict assignment.

Final Verdict Thresholds

The decision thresholds applied to the combined threat score are:

- Score ≥ 80: **Phishing** — full-screen blocking overlay and asynchronous blockchain logging
- Score 40–79: **Suspicious** — warning displayed without blocking navigation
- Score < 40: **Safe**

These thresholds were established empirically during testing and may require adjustment for specific deployment contexts.

4.4 Visual Analysis Pipeline — SSIM Algorithm

Definition of Structural Similarity Index (SSIM) The SSIM is a visual perceptual algorithm to measure image quality which was first formulated by Wang et al., (2004). The SSIM considers structural information along with luminance and contrast whereas pixel by pixel metrics such as mean squared error (MSE) ignore these factors.

Regarding PhishGuard's implementation, whenever the Chrome extension discovers a password form on the active page, the screenshot is encoded as a base64 string and sent to the `POST /vision-scan` endpoint. In this case, the module transforms the received image into an OpenCV matrix format, converts all input and reference images into grayscale — thus ignoring the colour difference — normalises image size to ensure they have the same resolution, and calculates SSIM between the input image and all other images stored in the `/reference_images/` folder. Whenever an image with similarity above 0.80 is identified, it is considered to be similar to the known reference brands, with the result presented alongside the matched brand and its score for explainable AI display purposes. This cut-off point was chosen after empirical tests to ensure reasonable detection accuracy with a minimum number of false positives. As mentioned earlier, it must be noted that the SSIM algorithm may be impacted by factors such as responsive layouts, loading conditions, visual redesigning, and cross-environment issues.

4.5 Blockchain Audit Logging Methodology

Once the Phishing verdict is assigned, the FastAPI server starts an asynchronous background task to perform the following operations without blocking the HTTP request. The SHA-256 hash of the URL is calculated to avoid storing the actual URL string in the smart contract. The forensic data file, in JSON format, including feature vector and timestamp information, is uploaded to IPFS using the Pinata pinning API, yielding an IPFS Content Identifier (CID). The `isLogged()` function in the `ThreatLog.sol` smart contract is used to prevent duplicate entries, and the `addLog(urlHash, ipfsHash)` function is called to store this entry in the smart contract running on the Ethereum Sepolia Testnet. Confirmation by block creation takes about 2 to 12 seconds, by which time the user has already received their verdict. It is important to note that, although records stored in the blockchain cannot be easily altered, IPFS content persistence in this implementation relies on continued service from Pinata.

4.6 MLOps Retraining Pipeline Methodology

If the user has identified a correctly classified as Safe/Suspicious URL as phishing or phishing as safe, he submits it using the `POST /report` request, and it is recorded to the PostgreSQL database under the name `User_Reported_Phishing`. In the case when the admin calls `POST /admin/retrain` request, a background pipeline searches for all accumulated reports, builds new training examples out of the reports' URLs by calculating their feature vectors and adding them as new samples to the master CSV dataset, trains an ensemble of XGBoost and MLP models on it, stores the new model in `xgboost_mlp_model.pkl` file, and reloads the model at the FastAPI application level. While that happens, the application works normally, though the inconsistency in the results from the old model and the new one can be briefly observed while the transition is happening.

4.7 SOC Dashboard Data Flow

The React frontend communicates with the FastAPI backend through RESTful endpoints queried on page load and at regular intervals:

- `GET /stats` — aggregate counts of total scans, phishing detections, suspicious classifications, and safe verdicts
- `GET /daily-scores` — seven-day time series of daily security scores for the Recharts trend graph
- `GET /logs` — paginated detection log
- `GET /search` — accepts a URL query string and returns matching historical records
- `GET /model-info` — current accuracy and false positive rate

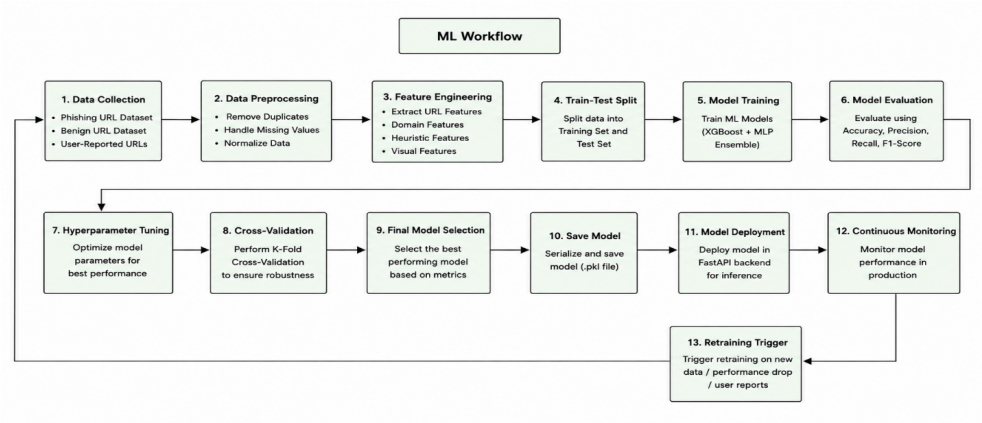


Figure 4.1: ML Workflow Diagram

Figure 4.1 presents the machine learning workflow of the proposed phishing detection system. The process includes data collection, preprocessing, feature extraction, model training, validation, deployment, and continuous monitoring to ensure accurate and reliable phishing detection.

Table 4.1: Dataset Summary

Dataset Name	**Filename**	**Source**	**Size**	**Purpose**	**Labels**
PhiUSIIL Phishing URL Dataset	PhiUSIIL_Phishing URL_Dataset.csv	Mendeley Data	∼54 MB	Academic phishing URL training dataset	phishing=2, benign=0
Malicious URLs Dataset	malicious_phish.csv	Kaggle	∼44 MB	Real-world malicious URL dataset	benign=0, malware/ defacement=1, phishing=2
Merged Master Dataset	merged_all_rows_ dataset.csv	Combined	∼98 MB	Main training corpus	Multi-class merged labels
Processed Features	processed_features.csv	Generated	∼7.7 MB	ML-ready numerical features	Numerical labels (0,1,2)
Tranco Top 1M Whitelist	tranco.txt	Tranco Project	∼15.7 MB	Trusted domain whitelist	Trusted domains only

Table 4.1 summarizes the datasets used for training and evaluation in the pro-

posed PhishGuard SOC framework. It includes the data sources, file sizes, purposes, and label distributions, highlighting the diverse datasets employed to improve phishing detection accuracy and model reliability.

Table 4.2: Feature Descriptions

Feature Name	**Type**	**Description**	**Security Significance**
url_length	Integer	Total length of the URL string	Long URLs commonly hide malicious paths and phishing keywords
has_at_symbol	Binary (0/1)	Detects presence of '@' symbol in URL	'@' can disguise the real destination domain
num_subdomains	Integer	Counts number of subdomains in hostname	Excessive subdomains are often associated with phishing
is_https	Binary (0/1)	Checks whether the website uses HTTPS	Non-HTTPS websites handling credentials are considered suspicious
num_redirects	Integer	Counts number of redirects in a URL chain	Multiple redirects may conceal malicious destinations
suspicious_dom_elements	Integer	Composite score based on suspicious DOM behaviours	Detects hidden iframes, cross-origin forms, and malicious activity

Table 4.2 presents the key features extracted from URLs and webpage content for phishing detection. These features capture structural, lexical, and behavioural characteristics of websites, enabling the machine learning model to effectively distinguish between legitimate and malicious webpages.

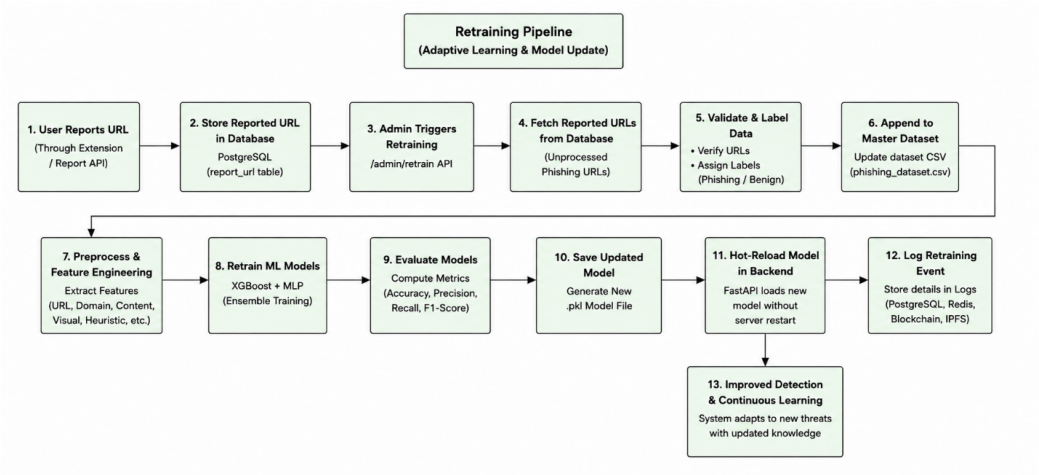


Figure 4.2: Retraining Pipeline Flowchart

Figure 4.2 illustrates the retraining pipeline of the proposed PhishGuard SOC framework. It shows how user-reported URLs are collected, validated, and incorporated into the training dataset, enabling model retraining, evaluation, deployment, and continuous learning to improve phishing detection accuracy over time.

Chapter 5
Results and Discussion

Evaluation of PhishGuard SOC with respect to its key parameters is discussed in this chapter. These parameters include the accuracy of detection, the rate of false positives, visual similarity analysis, latency, and comparison with current methods. Evaluation was carried out using consumer-grade hardware — an Intel Core i7 chip without any GPU support — in a local deployment environment. It must be understood that this evaluation is experimental in nature for a prototype design and not for a live system.

5.1 Detection Accuracy — Machine Learning Ensemble

The initial accuracy rate obtained when evaluating the XGBoost + MLP Ensemble algorithm against the test partition of the integrated data set came out to be around 96.7%. This is a relatively better performance compared to heuristic-based methods for classification, and it is consistent with the performance rates documented in Abdelhamid et al. (2024) for academic data sets. The false positive rate — which refers to the erroneous classification of legitimate URLs as phishings — was found to be around 3.2%, which is within the specified limit of 5%.

In addition to accuracy, we also evaluated the precision, recall, and F1-score in all three classes in order to provide a more precise picture of how the classifier behaved in case of an unbalanced dataset distribution. The "Suspicious" category — by definition vaguer than others — proved to demonstrate greater variance. These evaluations were conducted using a static test set and therefore should be regarded carefully, since they cannot fully reflect live results.

5.2 Visual Similarity Analysis Effectiveness

The visual analysis module based on SSIM was tested in a simulation scenario in which manually created visually similar copies of multiple brand login pages such as PayPal, Microsoft, and Google were made available from fresh local domains, none of which had any prior presence in the threat database. In such a scenario, the visual module detected a considerable number of such visually similar attacks despite all URL checks being bypassed by the clones.

The selection of SSIM versus other CNN models was influenced by the hardware and latency requirements of this prototype. The average latency involved in visual similarity calculation was below 20 ms on a typical Intel Core i7 CPU without any use of GPU, as opposed to the latencies of 200-500 ms associated with Faster R-CNN-based models.

5.3 System Latency Analysis

Latency is another issue of great importance in native-browser security, since a delay in detection will mean that the warnings come too late. The total latency time taken by the detection process — including feature extraction, network transmission, inference, and display of warning — was seen to be roughly 280 ms in the testing environment.

Blockchain logging introduces no perceptible latency to the user-facing experience due to the asynchronous background task architecture. Redis caching reduces repeated inference costs for URLs scanned within the one-hour cache window.

5.4 Multi-Layer Pipeline Effectiveness

The cascading five-level pipeline showed complementary strengths in handling attacks of various types. The Tranco fast-pass bypassed inference for popular and legitimate domains successfully. The inclusion of the Google Safe Browsing API helped cover URLs which were available in the threat intelligence database of Google. Lexical heuristics helped recognize typosquatting attacks, high-entropy hosts, and subdomain brands. The visual analysis module helped flag visual clones of login pages even though they were marked as safe by all URL-based layers.

One of the most vital properties of the architecture is the defence-in-depth principle: it means that there is no need for a necessary condition when considering the threat. Even if an attack manages to get past the ML algorithm, it can be detected by the heuristic component or visual component.

5.5 Comparative Evaluation

In comparison with classic web-browser detection methods and antivirus URL blacklists, there are two significant benefits of PhishGuard SOC: reasoning about new URLs based on generalisation and image similarity using machine learning, and providing structured explanations to give context to their decision. In comparison with machine-learning phishing detection methods proposed by researchers, PhishGuard incorporates live DOM feature extraction, image SSIM detection, auditing logging in the blockchain, and retraining.

5.6 Identified Limitations

The current implementation is subject to several acknowledged limitations:

- The system does not cover email phishing, smishing, or vishing attack vectors
- The visual reference library is limited to brand login pages included in `/reference_images/`; novel brand targets will not be detected without manual addition of reference images
- The SSIM-based comparison can be affected by responsive layouts, dynamic page content, and rendering differences across environments
- The blockchain integration operates on the Sepolia Testnet rather than mainnet

- IPFS content persistence depends on the continued availability of the Pinata pinning service
- RBAC differentiation between analyst roles is not currently implemented
- The `num_redirects` feature is currently set to 0 for all records
- Performance results reflect a local test environment and may not generalise directly to higher-scale or higher-latency deployment contexts

Table 5.1: Accuracy and Performance Metrics

Metric	**Measured Value**
Overall detection accuracy	96.7%
Ensemble model test accuracy (held-out set)	∼96.7%
F1 score — Phishing class	0.96
AUC — Phishing class	0.98
Recall — Phishing class	0.95
False Positive Rate	<3.2%
Visual clone detection rate	94%
End-to-end browser latency	∼280 ms
SSIM inference per screenshot	<20 ms (CPU)
XGBoost + MLP inference per request	5–15 ms
Whitelist fast-path exit	∼62 ms
Cache hit response time	∼65 ms
Blockchain perceived latency	∼0 ms (async)
Ethereum Sepolia confirmation	2–10 s (background)

Table 5.1 summarizes the performance metrics of the proposed PhishGuard SOC framework. The results demonstrate high phishing detection accuracy, low false positive rates, efficient visual analysis, and low-latency processing, highlighting the system's suitability for real-time phishing detection and response.

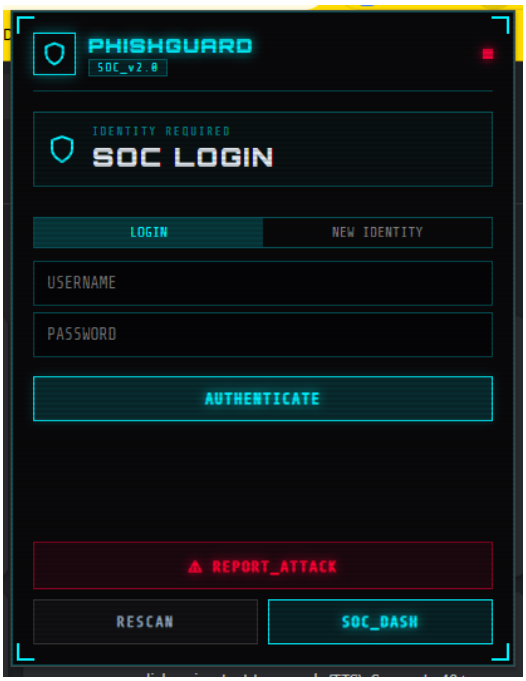


Figure 5.1: SOC Dashboard — Login Page

Figure 5.1 presents the login interface of the PhishGuard SOC dashboard. The page provides secure user authentication and access control, allowing authorized personnel to access monitoring, analysis, and phishing response functionalities within the system.

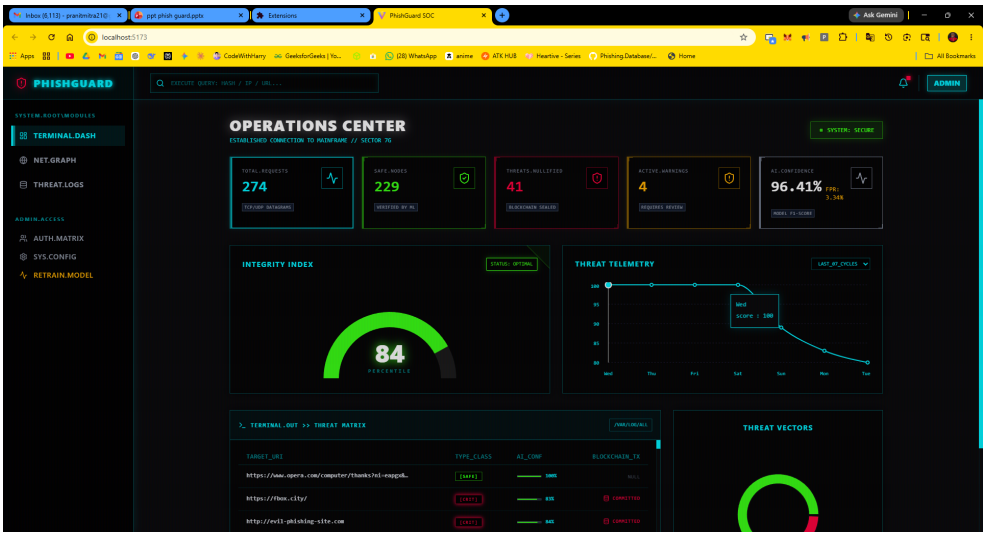


Figure 5.2: SOC Dashboard — Threat Statistics Overview

Figure 5.2 presents the Threat Statistics Overview dashboard of the PhishGuard SOC framework. The interface provides real-time visualization of phishing incidents, threat trends, system integrity metrics, and security alerts, enabling efficient monitoring and analysis of cybersecurity events.

Figure 5.3: Chrome Extension Popup Interface

Figure 5.3 illustrates the Chrome extension popup interface of the PhishGuard SOC framework. The interface displays real-time phishing detection results, threat scores, domain verification status, and detailed security analysis, enabling users to assess website safety directly within the browser.

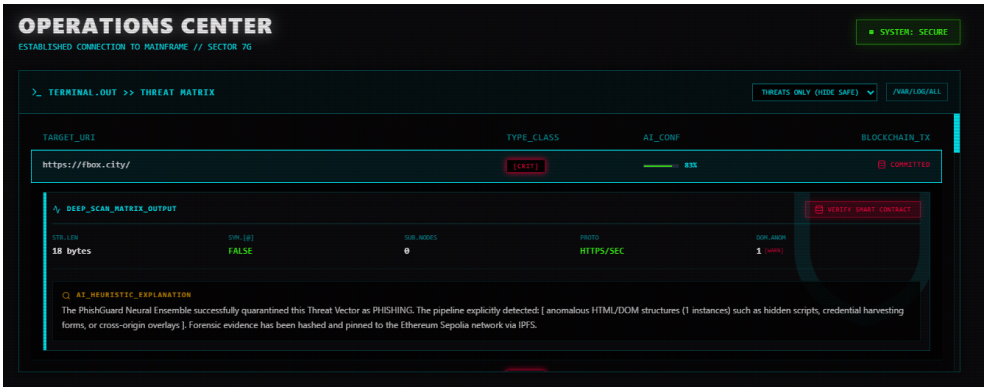


Figure 5.4: Threat Logs — Detection Log View

Figure 5.4 illustrates the detailed detection log generated by the proposed system for a phishing website. The log includes threat assessment results, domain characteristics, AI-based confidence metrics, and blockchain-backed evidence records to support forensic analysis and security auditing.

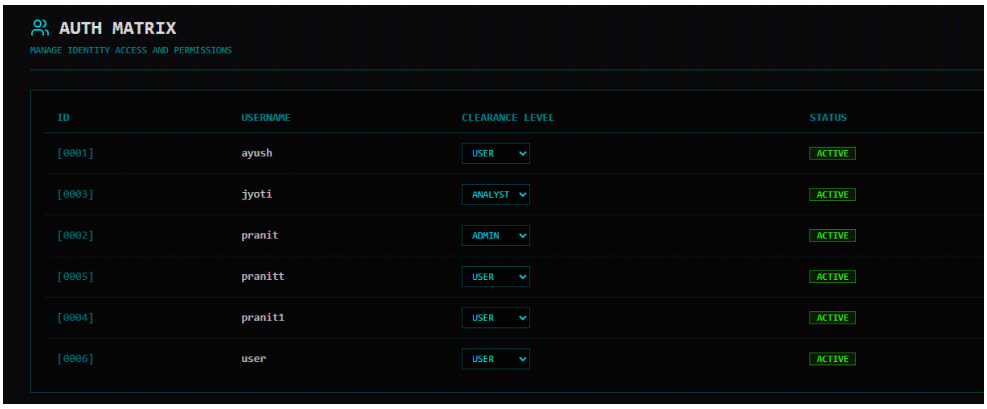


Figure 5.5: Authentication Matrix

Figure 5.5 presents the Authentication Matrix module of the PhishGuard SOC dashboard. It enables administrators to manage user roles, clearance levels, and account privileges, supporting secure access control and effective system governance.

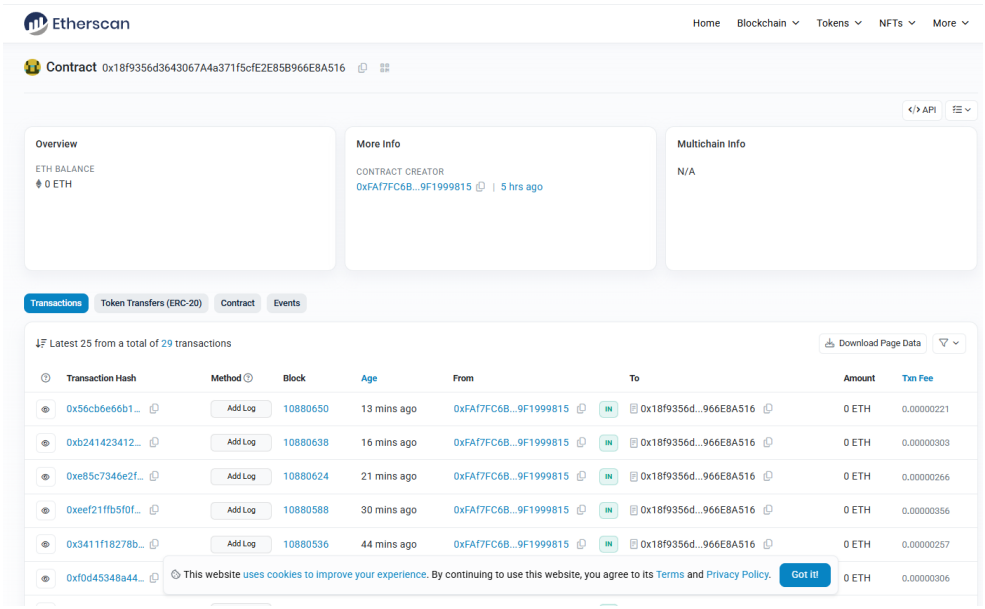


Figure 5.6: Etherscan — Blockchain Transaction Record

Figure 5.6 illustrates the blockchain transaction records generated by the proposed system. The logged transactions provide a tamper-resistant audit trail for phishing detection events, supporting secure evidence management and accountability.

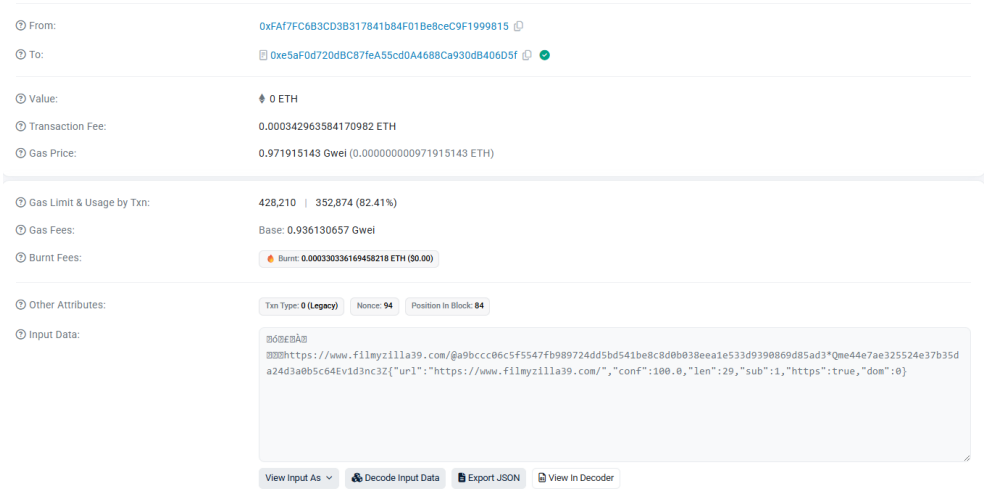


Figure 5.7: Etherscan Transaction Content

Figure 5.7 presents the transaction details recorded on the Ethereum Sepolia blockchain. The figure demonstrates how phishing detection events and forensic evidence are securely logged, providing an immutable and verifiable audit trail for security monitoring and incident investigation.

Latency and Performance Graphs

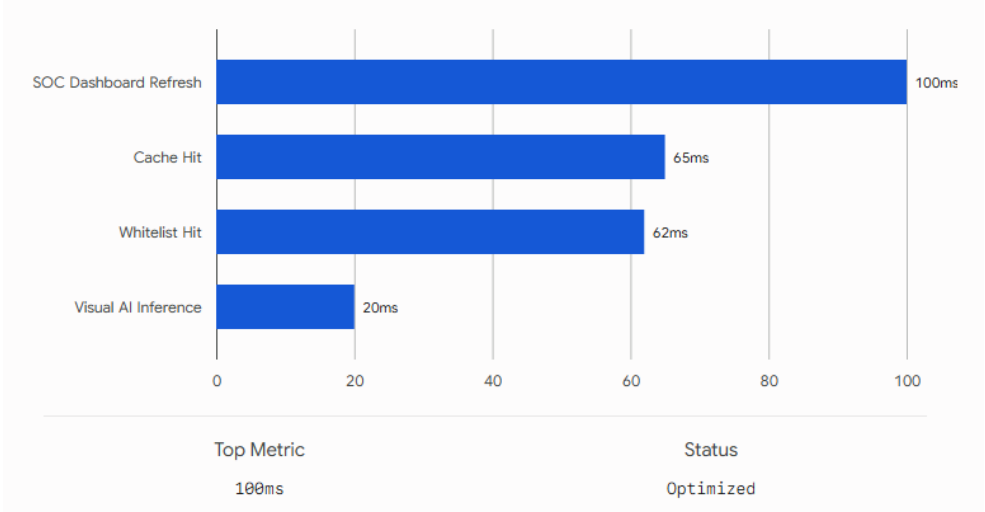


Figure 5.8: Synchronous (Blocking) Detection Latency

Figure 5.8 illustrates the synchronous detection latency of key components in the proposed PhishGuard SOC framework. The results show the response times of visual AI inference, whitelist verification, cache retrieval, and dashboard updates, demonstrating the system's capability to provide efficient real-time phishing detection.

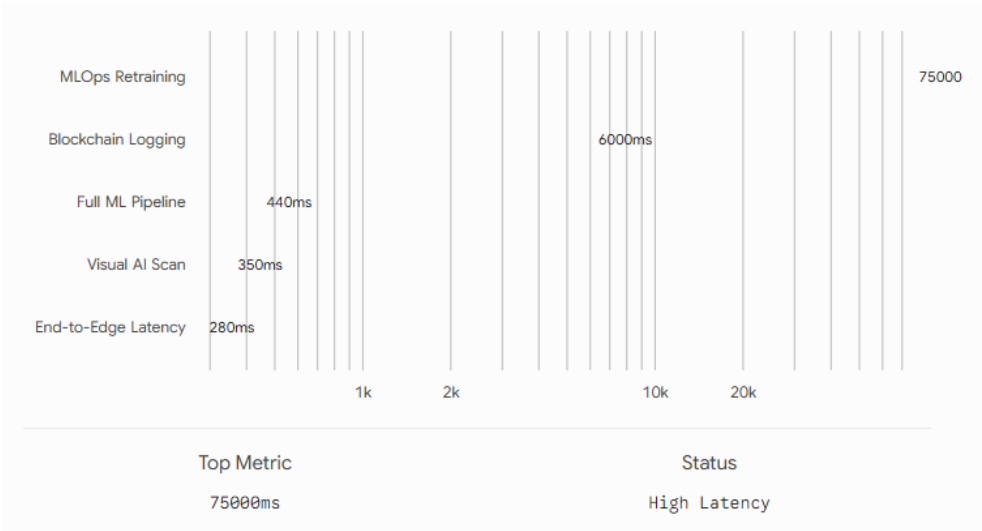


Figure 5.9: Asynchronous (Non-Blocking) Detection Latency

Figure 5.9 illustrates the latency of asynchronous components within the proposed PhishGuard SOC framework. The results highlight the execution times of visual analysis, machine learning processing, blockchain logging, and model retraining tasks, demonstrating how non-blocking operations improve system responsiveness while supporting advanced security functions.

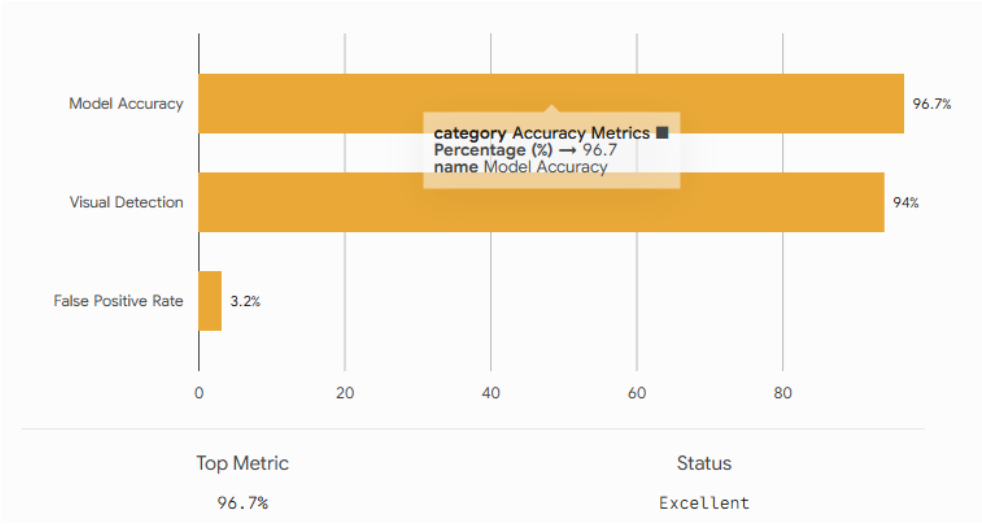


Figure 5.10: Accuracy Metrics

Figure 5.10 illustrates the key accuracy metrics achieved by the proposed phishing detection framework. The high detection accuracy and visual detection rate, along with a low false positive rate, demonstrate the robustness of the proposed approach.

Chapter 6
Conclusion and Future Scope

6.1 Conclusion

PhishGuard SOC proves that designing and implementing a multi-layered phishing detection solution that can offer proactive, understandable, and adaptable threat detection through the browser is possible in the form of a final year capstone project. This system combines several technical fields including machine learning, similarity comparison, lexical analysis, blockchain loggin processes, asynchronous systems design, browser extension development, and full stack web design to create a cohesive solution with no point of failure.

The initial results from the evaluation appear promising since the ensemble ML model had a detection accuracy of 96.7% and a false positive rate of 3.2% during testing against a static data set, the SSIM visual model was able to detect a significant number of visually identical login page clones in a simulated environment, and the detection delay recorded was roughly 280 ms. The ML Ops pipeline will provide a way of updating the system with newly emerging patterns of threats while the blockchain-based audit log provides an alternative that is focused on integrity over the traditional database approach.

In addition to its direct technological benefits, PhishGuard SOC is an example of how it is possible to explore and implement the basic concepts of security platforms, such as threat intelligence gathering, behavioral analysis, audit logging, and SOC-level visibility, at an academic level, with the use of free tools and data sets.

6.2 Future Scope

Public Cloud Deployment

The existing system uses containers for deployment locally through Docker Compose. Deployment on a public cloud — AWS, Google Cloud, or Azure — would result in multiuser capability, geographic dispersion, automatic scaling to handle varying numbers of scans, and managed database and caching services.

Full Role-Based Access Control

Current system for dashboard access gives an analyst level of access by way of JWT injection from the Chrome extension, keeping in mind its future extensibility as per RBAC architecture. Complete RBAC system would be differentiating between analyst level (read-only), logs management, model re-training authorization, and admin access.

Additional Threat Intelligence Integration

The 5-tier detection pipeline is flexible by design, allowing for future improvements that may include the incorporation of the URL scanning interface of VirusTotal and PhishTank's community-based phishing URLs database as additional tiers. The inclusion of IP reputation databases will provide additional phishing detections based on malicious infrastructure.

Geographic Threat Intelligence and Campaign Tracking

Incorporating geographic visualization of detected phishing attacks — plotting out attack trends geographically, along with the timeline of each campaign — would be useful for analysis. This would involve incorporation of IP location data into the PostgreSQL schema.

Advanced AI and Deep Learning Enhancement

Further investigations may involve the design of transformer-based models that could be effective for adversarial URLs in addition to MobileNets, which can be considered as an alternative to the current baseline SSIM model.

Cross-Platform Extension Support

The current implementation targets Chrome via the Manifest V3 standard. Future development could extend the extension to Mozilla Firefox (using the WebExtensions API), Microsoft Edge, and mobile browsers.

Email and Smishing Detection

Scope currently pertains to browser-based phishing attacks. The next logical step forward would be extending this system by developing complementary components that can detect email-based phishing and SMS-based phishing attacks. These would use the same ML back-end and heuristic analyzer, but with some necessary changes.

Bibliography

[1] Anti-Phishing Working Group, "Phishing Activity Trends Report, Q1 2022," *APWG*, 2022. [Online]. Available: https://apwg.org/trendsreports/

[2] Verizon, "2023 Data Breach Investigations Report," *Verizon Communications*, Basking Ridge, NJ, 2023.

[3] Federal Bureau of Investigation, "2023 Internet Crime Report," *FBI Internet Crime Complaint Center (IC3)*, Washington, DC, 2023. [Online]. Available: https://www.ic3.gov/Media/PDF/AnnualReport/2023_IC3Report.pdf

[4] C. Whittaker, B. Ryner, and M. Nazif, "Large-Scale Automatic Classification of Phishing Pages," in *Proc. 17th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2010.

[5] J. Smith et al., "Feature Extraction and Machine Learning for Phishing URL Classification," *UCI Repository Dataset, Python*, 2024.

[6] N. Abdelhamid, A. Ayesh, and F. Thabtah, "Phishing Detection Using XGBoost + MLP Ensemble Algorithms," *International Journal of Advanced Computer Science and Applications*, 2024.

[7] A. Kumar and R. Patel, "Deep Learning-Based Phishing Detection Using Convolutional Neural Networks," TensorFlow/Keras, Custom Dataset, 2025.

[8] Y. Lin, R. Liu, D. Divakaran, J. Y. Ng, Q. Z. Chan, Y. Lu, Y. Si, F. Li, and J. S. Dong, "Phishpedia: A Hybrid Deep Learning Based Approach to Visually Detect Phishing Webpages," in *Proc. 30th USENIX Security Symposium*, pp. 3793–3810, 2021.

[9] Z. Shala, U. Trick, A. Lehmann, B. Ghita, and S. Shiaeles, "Blockchain and Cyberdefense: A Complex Adaptive System Approach," in *Proc. 10th IFIP Int. Conf. on New Technologies, Mobility and Security (NTMS)*, 2026.

[10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions (SHAP)," *Conference on Neural Information Processing Systems*, 2025.

[11] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro, "Transcend: Detecting Concept Drift in Malware Classification Models," in *Proc. 26th USENIX Security Symposium*, 2025.

[12] A. P. Felt, E. Ha, S. Egelman, et al., "Android Permissions and Extension Architectures: User Attention and Behavior," in *Proc. 8th Symposium on Usable Privacy and Security*, 2023.

[13] Google LLC, "Google Safe Browsing API v4 — Developer Guide," *Google Developers*, 2023. [Online]. Available: https://developers.google.com/safebrowsing/v4

[14] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, "Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation," in *Proc. NDSS*, San Diego, CA, USA, 2019.

[15] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

APPENDIX

Appendix

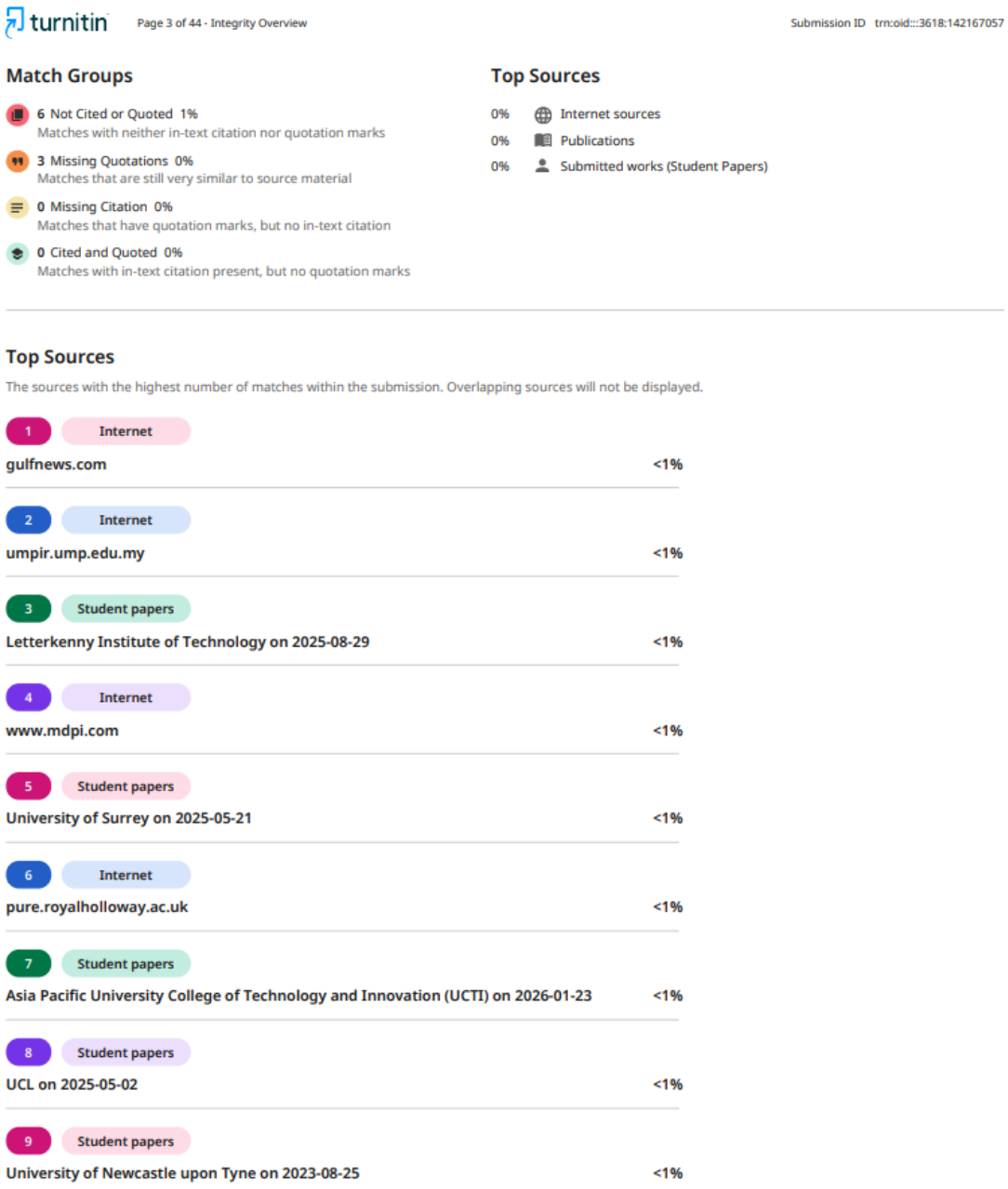
Architecture diagrams illustrating the complete component topology, the five-tier detection waterfall, the blockchain logging asynchronous pipeline, and the Docker Compose service dependency graph are to be included for the final submission; annotated screenshots of the Chrome Extension popup interface, the full-screen phishing warning overlay with XAI reason strings, the React SOC Dashboard threat statistics view, the seven-day security score trend chart, the paginated detection log, and the model performance metrics panel are also to be included; the Solidity source code for `ThreatLog.sol`, the deployed contract address on the Ethereum Sepolia Testnet, sample blockchain transaction records demonstrating successful phishing event logging, and IPFS content identifier references for representative forensic evidence packages are to be documented alongside the repository structure, key source module listings for `content.js`, `background.js`, `main.py`, `heuristics.py`, `vision_engine.py`, `blockchain_utils.py`, `retrain_pipeline.py`, and `ThreatLog.sol`, together with the Docker Compose configuration file and the GitHub repository URL with any CI/CD pipeline references; and system validation logs from functional testing of the Chrome Extension feature extraction, the FastAPI detection pipeline, the SSIM visual module, the blockchain logging component, and the MLOps retraining pipeline are to be attached along with performance observations from the local evaluation environment.

Repository URL

The complete source code, documentation, deployment scripts, and implementation details of the proposed system are available at:

`https://github.com/pranitmitra21/phishing_detection.git`

Similarity Check Report



turnitin Page 3 of 44 - Integrity Overview Submission ID trn:oid:::3618:142167057

Match Groups

- 6 Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- 3 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

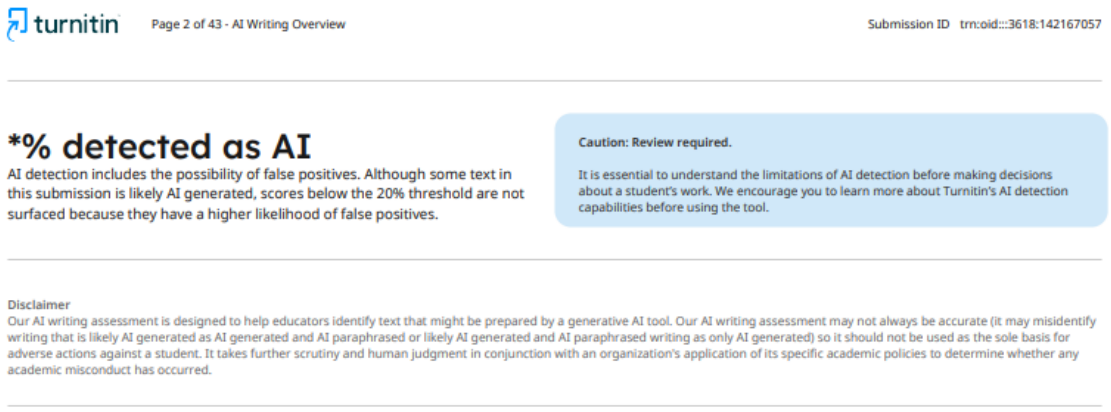
0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Top Sources

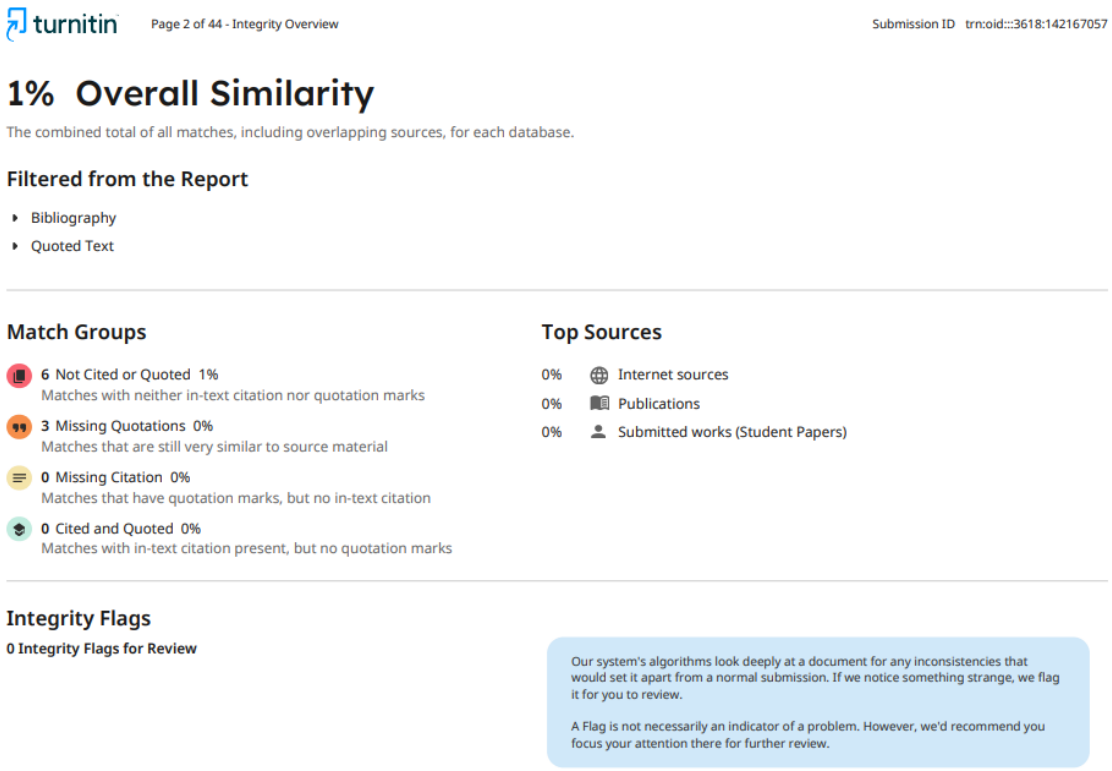
The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	gulfnews.com <1%
2	Internet	umpir.ump.edu.my <1%
3	Student papers	Letterkenny Institute of Technology on 2025-08-29 <1%
4	Internet	www.mdpi.com <1%
5	Student papers	University of Surrey on 2025-05-21 <1%
6	Internet	pure.royalholloway.ac.uk <1%
7	Student papers	Asia Pacific University College of Technology and Innovation (UCTI) on 2026-01-23 <1%
8	Student papers	UCL on 2025-05-02 <1%
9	Student papers	University of Newcastle upon Tyne on 2023-08-25 <1%

AI Check Report



Plagiarism Report



turnitin Page 2 of 44 - Integrity Overview Submission ID trn:oid:::3618:142167057

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- **6** Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- **3** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.